

WEEK1 relational-algebra-notes

Relational Algebra — Part 1: Basics & Core Operators

Study notes based on the Stanford relational algebra lecture (Video 1 of 2). Covers the query language fundamentals and the most common operators.

1. What Is Relational Algebra?

Relational algebra is a *formal language* — a mathematical algebra that forms the theoretical underpinning of implemented query languages like **SQL**.

Two key properties to remember:

- It **operates on relations** (tables).
- It **produces relations as a result**.

Because the output of a query is itself a relation, you can:

- Feed that result into **another query**, or
- **Combine** it with other existing relations.

This *closure property* (input is a relation, output is a relation) is what makes operators **composable** — you can stack them freely.

Context — "relation" vs "table": A *relation* is the formal term for what SQL calls a *table*. A *tuple* = a row; an *attribute* = a column; the *schema* = the set of attribute names (the table's column structure).

2. The Example Database

All examples use a simple **college admissions** database with three relations. Keep this picture in mind throughout.

Schemas

College

Attribute	Meaning
cName (<i>key</i>)	College name
state	State the college is in
enrollment	Number of students enrolled

Student

Attribute	Meaning
sID (<i>key</i>)	Student ID
sName	Student name
GPA	Grade point average
sizeHS	Size of the high school attended

Apply

Attribute	Meaning
sID (<i>part of key</i>)	Student ID (who is applying)
cName (<i>part of key</i>)	College applied to
major (<i>part of key</i>)	Major applied for
decision	Application outcome (e.g., Y = accept, N = no, R = reject)

Keys are underlined in the lecture (shown in bold here). A **key** is an attribute (or set of attributes) whose value is *guaranteed to be unique* across the relation.

Key assumptions for this database:

- College names are unique → cName is the key of College.
- Student IDs are unique → sID is the key of Student.
- A student applies to a given college for a given major **only once** → the key of Apply is the *combination* (sID, cName, major). This is called a **composite key**.

Sample data — use these tables to trace every example below

College

cName	state	enrollment
Stanford	CA	21000
Berkeley	CA	36000
MIT	MA	11000
Cornell	NY	21000
Harvard	MA	22000

Student

sID	sName	GPA	sizeHS
123	Amy	3.9	800
234	Bob	3.6	1500
345	Craig	3.5	500
456	Doris	3.9	1000
567	Edward	2.9	2000
678	Fay	3.8	200
789	Gary	3.4	800

Apply

sID	cName	major	decision
123	Stanford	CS	Y
123	Stanford	EE	N
123	Berkeley	CS	Y
234	Berkeley	biology	N
234	Stanford	CS	R
345	MIT	bioengineering	Y
345	Cornell	bioengineering	N
345	Cornell	CS	Y
345	Cornell	EE	R
456	Stanford	CS	N
678	Stanford	history	Y
789	Cornell	CS	R

3. The Simplest Query: A Relation Name

The simplest valid relational algebra expression is just the **name of a relation**:

Student

Running this returns a **copy of the entire Student relation** (the 7-row table above). Straightforward — but it's the base case that every more complex expression builds on.

From here, we apply **operators** to **filter**, **slice**, and **combine** relations.

4. Select Operator — σ (picks ROWS)

The **select** operator picks **certain rows (tuples)** out of a relation based on a condition.

- **Symbol:** σ (Greek sigma)
- **Subscript:** the *condition* used to filter rows
- **Form:** $\sigma_{\text{condition}}$ (Relation)

Memory aid: Select $\rightarrow$ Sigma $\rightarrow$ picks rows (horizontal slice).

Example 1 — single condition

Find students whose GPA is greater than 3.7:

$\sigma_{\text{GPA} > 3.7}$ (Student)

Result: keep rows where GPA > 3.7. Amy (3.9), Doris (3.9), Fay (3.8) qualify.

sID	sName	GPA	sizeHS
123	Amy	3.9	800
456	Doris	3.9	1000
678	Fay	3.8	200

Example 2 — two conditions (AND)

Combine conditions using the logical AND operator $\wedge$ (a caret $\wedge$):

Find students with GPA > 3.7 **and** high school size < 1000:

$\sigma_{\text{GPA} > 3.7 \wedge \text{sizeHS} < 1000}$ (Student)

Result: of the three high-GPA students, Doris's sizeHS is 1000 (not strictly less), so she drops out.

sID	sName	GPA	sizeHS
123	Amy	3.9	800
678	Fay	3.8	200

Example 3 — condition on another relation

Find applications to Stanford for a CS major:

$\sigma_{(cName = 'Stanford' \wedge major = 'CS')}$ (Apply)

Result:

sID	cName	major	decision
123	Stanford	CS	Y
234	Stanford	CS	R
456	Stanford	CS	N

General form

$\sigma_{condition} (Relation) \rightarrow$ subset of the relation's rows

Conditions can use comparisons ($=$, $\neq$, $<$, $>$, $\leq$, $\geq$) and be combined with logical operators $\wedge$ (AND), $\vee$ (OR), and $\neg$ (NOT).

5. Project Operator — π (picks COLUMNS)

The **project** operator picks **certain columns (attributes)** from a relation. Where select slices horizontally (rows), project slices vertically (columns).

- **Symbol:** π (Greek pi)
- **Subscript:** the *list of column names* to keep
- **Form:** $\pi_{(attr1, attr2, \dots)} (Relation)$

Memory aid: Project $\rightarrow$ Pi $\rightarrow$ Picks columns.

Example

Get just the student IDs and decisions from all applications:

$\pi_{(sID, decision)}$ (Apply)

This keeps **all the tuples** of Apply, but **only the sID and decision columns** — *and* eliminates duplicates (see §7).

Result: the 12 application rows collapse to these distinct (sID, decision) pairs:

sID	decision
123	Y
123	N
234	N
234	R
345	Y
345	N
345	R
456	N
678	Y
789	R

Notice student 123 originally had three rows (Y, N, Y); after projecting away cName and major, the two (123, Y) tuples become duplicates and merge into one.

General form

$\pi_{\text{(list of attributes)}}(\text{Relation})$

6. Composing Operators (Rows AND Columns)

Because every query produces a relation, operators can be **nested**. This lets us pick *some rows* **and** *some columns* at once.

Find the IDs and names of students with GPA > 3.7:

$\pi_{\text{(sID, sName)}}(\sigma_{\text{(GPA > 3.7)}}(\text{Student}))$

Read inside-out:

Step 1 — $\sigma_{\text{(GPA > 3.7)}}(\text{Student}) \rightarrow$ rows with high GPA:

sID	sName	GPA	sizeHS
123	Amy	3.9	800

sID	sName	GPA	sizeHS
456	Doris	3.9	1000
678	Fay	3.8	200

Step 2 — $\pi_{\text{(sID, sName)}} (...) \rightarrow$ keep only the ID and name columns:

sID	sName
123	Amy
456	Doris
678	Fay

The *real* general form (the "slight deception")

The lecture admits the earlier definitions were simplified. The operators don't just apply to a *relation name* — they apply to **any relational algebra expression**:

$\sigma_{\text{condition}} (<\text{any expression}>)$
 $\pi_{\text{(attributes)}} (<\text{any expression}>)$

You can compose as deeply as you like — select over project over select, etc. Use **parentheses** to keep big expressions clear.

7. Duplicate Elimination (Sets vs. Bags)

Consider listing every major applied for and its decision:

$\pi_{\text{(major, decision)}} (\text{Apply})$

In the raw 12-row Apply, the pair (CS, Y) occurs multiple times (123→Stanford CS Y *and* 123→Berkeley CS Y *and* 345→Cornell CS Y), and (CS, R) occurs twice (234→Stanford and 789→Cornell).

Relational algebra eliminates duplicates automatically. Each distinct result appears **exactly once**.

major	decision
CS	Y
EE	N
biology	N
CS	R

major	decision
bioengineering	Y
bioengineering	N
EE	R
CS	N
history	Y

12 raw projected rows → **9 distinct rows**.

Model	Duplicates?	Used by
Set semantics	Eliminated	Relational algebra (this course)
Multiset / Bag semantics	Kept	SQL

Important real-world note: SQL is based on **multisets (bags)** and does **not** remove duplicates by default — you must write `SELECT DISTINCT`. A bag-based relational algebra exists too, but these notes use **set** semantics throughout.

8. Cross-Product — $\times$ (Cartesian Product)

The first operator that **combines two relations**. Also called the **Cartesian product**.

What it does: "glues" two relations together.

- **Resulting schema** = the **union** of the two relations' schemas (all columns from both).
- **Resulting contents** = **every combination** of one tuple from the first with one tuple from the second.

Abstract illustration

Suppose $R_1(A, B)$ and $R_2(C, D)$:

R_1

A	B
a_1	b_1

A	B
a ₂	b ₂
R ₂	
C	D
c ₁	d ₁
c ₂	d ₂

$R_1 \times R_2$ — every row of R_1 paired with every row of R_2 ($2 \times 2 = 4$ rows, 4 columns):

A	B	C	D
a ₁	b ₁	c ₁	d ₁
a ₁	b ₁	c ₂	d ₂
a ₂	b ₂	c ₁	d ₁
a ₂	b ₂	c ₂	d ₂

Size of the result

If Student has **S** tuples and Apply has **A** tuples, then:

Student $\times$ Apply $\rightarrow S \times A$ tuples

In our database, that's $7 \times 12 = \mathbf{84}$ **tuples**, each with **8 attributes**.

Schema with eight attributes — and naming clashes

Both Student and Apply contain an sID attribute. To disambiguate, columns are **prefixed with the relation name**:

Student.sID vs Apply.sID

Concrete mini cross-product

To keep the table viewable, let's cross-product just the first 2 rows of each relation:

Student (first 2 rows)

sID	sName	GPA	sizeHS
123	Amy	3.9	800
234	Bob	3.6	1500

Apply (first 2 rows)

sID	cName	major	decision
123	Stanford	CS	Y
123	Stanford	EE	N

Student $\times$ Apply ($2 \times 2 = 4$ rows, 8 columns):

Student.sID	Student.sName	GPA	sizeHS	Apply.sID	Apply.cName	major	decision
123	Amy	3.9	800	123	Stanford	CS	Y
123	Amy	3.9	800	123	Stanford	EE	N
234	Bob	3.6	1500	123	Stanford	CS	Y
234	Bob	3.6	1500	123	Stanford	EE	N

Why bother with cross-product? Look at row 3: it pairs *Bob* (sID 234) with *Amy's* application (sID 123) — meaningless. On its own, cross-product produces lots of nonsense pairings. Its power appears when **combined with select** to keep only the meaningful combinations (the ones where Student.sID = Apply.sID).

9. Worked Example: Cross-Product + Select

Goal (in English): Find the **names and GPAs** of students with **high school size > 1000** who **applied to CS** and were **rejected**.

We need data from both Student and Apply, so start with their cross-product:

Student $\times$ Apply

This gives the 84-row, 8-attribute relation. Now filter it down with a big selection:

Step 1 — match real pairings only (the same student in both halves):

$\sigma_{\text{Student.sID} = \text{Apply.sID}}(\text{Student} \times \text{Apply})$

Without this, we'd keep nonsensical rows pairing a student with someone *else's* application.

Step 2 — add the remaining filters:

```
 $\sigma_{\text{(Student.sID = Apply.sID}} \\ \wedge \text{sizeHS} > 1000 \\ \wedge \text{major} = \text{'CS'} \\ \wedge \text{decision} = \text{'R'}} \\ (\text{Student} \times \text{Apply})$ 
```

Tracing through our data:

- $\text{sizeHS} > 1000$ keeps Bob (1500) and Edward (2000).
- $\text{major} = \text{'CS'} \wedge \text{decision} = \text{'R'}$ keeps (234, Stanford, CS, R) and (789, Cornell, CS, R).
- Joining on sID: only Bob (234) satisfies both halves.

Intermediate table (after Step 2):

Student.sID	sName	GPA	sizeHS	Apply.sID	cName	major	decision
234	Bob	3.6	1500	234	Stanford	CS	R

Step 3 — keep only name and GPA:

```
 $\pi_{\text{(sName, GPA)}} ( \\ \sigma_{\text{(Student.sID = Apply.sID}} \\ \wedge \text{sizeHS} > 1000 \\ \wedge \text{major} = \text{'CS'} \\ \wedge \text{decision} = \text{'R'}} \\ (\text{Student} \times \text{Apply}) \\ )$ 
```

Final result:

sName	GPA
Bob	3.6

Note how essential the Student.sID = Apply.sID "join condition" is — it's what makes the combination meaningful.

10. Natural Join — ⋈

The pattern above (cross-product + equality on shared attributes) is so common that relational algebra provides a dedicated operator: the **natural join**.

- **Symbol:** ⋈ (a bow tie — searchable in most text editors)

What the natural join does (three things):

1. Performs a **cross-product** of the two relations.
2. **Enforces equality** on **all attributes that share the same name** in both relations.
3. **Removes the duplicate columns** (keeps one copy of each shared attribute, since the values are guaranteed equal anyway).

Why duplicates can be dropped: If Student.sID *must equal* Apply.sID for a row to survive, keeping both columns is redundant — they're always identical. So the join keeps a single sID column.

Abstract illustration

Suppose $R_1(A, B)$ and $R_2(B, C)$ share attribute B:

R_1

A	B
a_1	b_1
a_2	b_2

R_2

B	C
b_1	c_1
b_3	c_2

$R_1 \bowtie R_2$ — only b_1 matches, and the duplicate B column collapses to one:

A	B	C
a_1	b_1	c_1

Edge case: If two relations share **no** attribute names, the natural join degenerates into the plain cross-product (no equality to enforce, no columns to drop).

Concrete: Student ⋈ Apply

Both share sID, so the join keeps only rows where sID matches, and outputs **7 attributes** (not 8):

sID	sName	GPA	sizeHS	cName	major	decision
123	Amy	3.9	800	Stanford	CS	Y
123	Amy	3.9	800	Stanford	EE	N
123	Amy	3.9	800	Berkeley	CS	Y
234	Bob	3.6	1500	Berkeley	biology	N
234	Bob	3.6	1500	Stanford	CS	R
345	Craig	3.5	500	MIT	bioengineering	Y
345	Craig	3.5	500	Cornell	bioengineering	N
345	Craig	3.5	500	Cornell	CS	Y
345	Craig	3.5	500	Cornell	EE	R
456	Doris	3.9	1000	Stanford	CS	N
678	Fay	3.8	200	Stanford	history	Y
789	Gary	3.4	800	Cornell	CS	R

Edward (567) is gone — he doesn't appear in Apply, so no natural-join row exists for him.

Same worked query, rewritten with natural join

```
π_(sName, GPA) (  
  σ_(sizeHS > 1000 ∧ major = 'CS' ∧ decision = 'R')  
  (Student ⋈ Apply)  
)
```

Trace: apply the σ filter to the joined table above — only row (234, Bob, 3.6, 1500, Stanford, CS, R) qualifies — then project sName, GPA:

sName	GPA
Bob	3.6

Cleaner than the cross-product version. **Setting up your schemas with consistent attribute names makes natural join very powerful.**

Joining three relations — adding a College filter

New requirement: only applications to colleges with **enrollment > 20,000**.

So far we used Student and Apply but not College. Bring it in by joining all three:

```
 $\pi_{\text{(sName, GPA)}} ($   
   $\sigma_{\text{(sizeHS > 1000 \wedge major = 'CS' \wedge decision = 'R' \wedge enrollment > 20000)}}$   
   $(\text{Student} \bowtie \text{Apply} \bowtie \text{College})$   
)
```

- Student $\bowtie$ Apply matches on shared sID.
- ... $\bowtie$ College matches on shared cName (both Apply and College have cName).

Student $\bowtie$ Apply $\bowtie$ College (8 attributes — sID, sName, GPA, sizeHS, cName, major, decision, state, enrollment). Filtering with the new condition: Bob applied to Stanford (enrollment 21000 > 20000) — still qualifies. **Result unchanged for our data:**

sName	GPA
Bob	3.6

Associativity: Natural join is technically a *binary* operator, but it's **associative**, so people usually write $A \bowtie B \bowtie C$ without parentheses. Being pedantic, you *could* write $(A \bowtie B) \bowtie C$.

Natural join adds no expressive power

The natural join is **convenient notation** — it doesn't let you express anything you couldn't already express with cross-product, select, and project.

General rewrite of $E1 \bowtie E2$:

```
 $E1 \bowtie E2 \equiv \pi_{\text{(schema(E1) \cup schema(E2))}} ($   
   $\sigma_{\text{(E1.A1 = E2.A1 \wedge E1.A2 = E2.A2 \wedge \dots)}} ($   
     $E1 \times E2$   
  )  
)
```

)
)

Where:

- $\text{schema}(E1) \cup \text{schema}(E2)$ is a **true set union** — shared attribute names appear **once** (this achieves the duplicate-column removal).
 - $A1, A2, \dots$ are the attributes that share the same name in both expressions (this achieves the equality enforcement).
-

11. Theta Join — $\bowtie_{\theta}$

The **theta join** combines two expressions with an **arbitrary condition θ** (theta).

- **Symbol:** $\bowtie$ with subscript θ
- **Form:** $E1 \bowtie_{\theta} E2$
- θ is a condition in the same style as a select condition.

Definition (it's simple): theta join = apply the condition θ to the cross-product.

$$E1 \bowtie_{\theta} E2 \equiv \sigma_{\theta}(E1 \times E2)$$

Mini example

Pair each student with each application **only when** the student's GPA is high *and* the application is to Stanford:

$\text{Student} \bowtie_{(\text{Student.sID} = \text{Apply.sID} \wedge \text{GPA} \geq 3.8 \wedge \text{cName} = \text{'Stanford'})} \text{Apply}$

Walking through: only Amy (3.9) and Fay (3.8) meet the GPA bar; of those, both have Stanford applications.

Student.sID	sName	GPA	sizeHS	Apply.sID	cName	major	decision
123	Amy	3.9	800	123	Stanford	CS	Y
123	Amy	3.9	800	123	Stanford	EE	N
678	Fay	3.8	200	678	Stanford	history	Y

Theta join keeps **both** sID columns (no auto-dedup, unlike natural join).

Like natural join, theta join **adds no expressive power** — it's an abbreviation.

Why it matters in practice: Most **database management systems implement the theta join as their fundamental operation** for combining relations: *take two relations, form all tuple combinations, keep only those passing condition θ* . When database practitioners say "**join**," they usually mean **theta join**.

Distinction to remember:

- **Natural join** matches automatically on *equal, same-named* attributes and removes duplicate columns.
 - **Theta join** uses *any explicit condition* you supply (e.g., $\text{Student.GPA} > 3.5$, $A.x \leq B.y$) and does **not** auto-remove columns.
-

12. Summary

Relational algebra is a **formal language** that **operates on relations and produces relations**. The simplest query is just a relation name; **operators** then filter, slice, and combine relations.

Operator	Symbol	Purpose	Adds expressive power?
Select	σ	Pick rows matching a condition	Yes (core)
Project	π	Pick columns (attributes)	Yes (core)
Cross-product	$\times$	Combine every pair of tuples from two relations	Yes (core)
Natural join	$\bowtie$	Combine relations, auto-matching same-named attributes & dropping	No (abbreviation)

Operator	Symbol	Purpose	Adds expressive power?
		duplicate columns	
Theta join	$\bowtie_{\theta}$	Cross-product filtered by an arbitrary condition θ	No (abbreviation)

Key takeaways:

- Output relations can feed into further queries $\rightarrow$ operators **compose**.
- Relational algebra uses **set semantics** $\rightarrow$ **duplicates are eliminated** (unlike SQL's bags).
- σ picks rows, π picks columns; nest them to do both.
- $\times$ is rarely useful alone but powerful with σ .
- $\bowtie$ and $\bowtie_{\theta}$ *are _conveniences* rewritable via $\times + \sigma + \pi$.
- In real database systems, "join" usually means **theta join**.

Quick symbol reference

Symbol	Name	Meaning
σ	sigma	select (rows)
π	pi	project (columns)
$\times$	times	cross-product
$\bowtie$	bow tie	natural join
$\bowtie_{\theta}$	bow tie + theta	theta join
$\wedge \vee \neg$	caret / etc.	AND, OR, NOT
$\cup$	union	set union (on schemas/relations)

Next video (Part 2): additional relational algebra operators and alternative notations.